

# Load, Concurrency & Security Test Report

Deployed to Floci EKS and driven under concurrent load, chaos and attack. Everything below was executed — nothing inferred. Where something could not be tested on Floci, it says so. 2026-08-01/02 · Kong → 6 services → 5 Postgres + Redis + Kafka + real ALB

## Result

|                                            | Found                                                                           | After the fix                  |
|--------------------------------------------|---------------------------------------------------------------------------------|--------------------------------|
| Deployment                                 | ✓ 14 pods, 7 PVCs bound, 5 Kafka topics                                         | —                              |
| Saga (single order)                        | ✓ PENDING → PAID in ~8s                                                         | —                              |
| <b>Concurrency (40 orders vs 10 units)</b> | ● <b>OVERSOLD — 18 sold from 10</b>                                             | ✓ exactly 10                   |
| Kong at default limits                     | ● <b>OOMKilled ×3</b>                                                           | ✓ 1Gi                          |
| <b>Authentication</b>                      | ● <b>5 of 6 services had none — POST /api/orders with no token returned 200</b> | ✓ 403                          |
| <b>Object-level authz</b>                  | ● <b>any caller could order in anyone's name</b>                                | ✓ identity from the token      |
| Pod kill under traffic                     | ● 502 + five 5s timeouts                                                        | ✓ 130/130 → 200                |
| <b>Saga replies under burst</b>            | ● <b>4 of 20 orders stranded at PENDING, silently</b>                           | ✓ PENDING=0                    |
| Containers as root                         | ● 6 services + redis                                                            | ✓ non-root, drop: ALL          |
| Kong Admin API                             | ● unauthenticated on 0.0.0.0:8001                                               | ✓ loopback only                |
| Rate limit with 3 Kong pods                | ● 100/min silently became 300/min                                               | ✓ shared Redis counter         |
| NetworkPolicy                              | ● none deployed by the scripts                                                  | ✓ 17, attacker pod blocked 4/4 |
| ALB target sync                            | ● one-time snapshot; dead IP after any restart                                  | ✓ follows pod kills in ~15s    |
| CI smoke test                              | ● ran entirely without a token                                                  | ✓ 29/29 incl. denial checks    |

## 1. Deployment

```
mvn -B -T 1C package -DskipTests # 6 jars
docker build → docker save → ctr images import # into k3s containerd
kubectl apply -f k8s/{namespace,services,postgres,redis,kafka,kong}
```

| Step                              | Result                       |
|-----------------------------------|------------------------------|
| 7 PVCs (5 Postgres, Redis, Kafka) | ✓ all Bound — k3s local-path |
| Kafka topics                      | ✓ 5 created by the init Job  |
| 6 microservices                   | ✓ Running                    |

KAFKA\_AUTO\_CREATE\_TOPICS\_ENABLE: "false", so the init Job is mandatory. It takes ~2 min waiting for Kafka; the deploy must not race ahead of it.

## ● Kong OOMKilled at the shipped limits

Last State: Terminated • Reason: OOMKilled • Restart Count: 3

Limits: memory 512Mi

Kong's worker processes were killed repeatedly ( worker process exited on signal 9 ), so **every request through the gateway failed** while the services themselves were healthy — user-service returned {"status": "UP"} on a direct port-forward.

The node was only at 45% memory, so this is Kong's own limit being too low, not node pressure.

**Fix applied:** limits.memory 512Mi → 1Gi , requests 256Mi → 512Mi . Stable after that, /api/products returned 200.

## 2. Functional test — the saga works

```
POST /api/users/register          → 200
POST /api/users/login             → 200 (JWT, 196 chars)
POST /api/products                → 200
POST /api/inventory/1/restock {qty:500} → 200
POST /api/orders                  → 200  status=PENDING
... 8 seconds ...
GET /api/orders/1                 → 200  status=PAID
```

✅ The full choreography ran: order.created → inventory reserved → inventory.updated → payment charged → payment.processed → PAID .

## 3. ● Concurrency test — overselling confirmed

**Setup:** a fresh product with **exactly 10 units**, then 40 simultaneous orders of 1 unit each.

```
curl POST /api/inventory/$PID/restock -d '{"quantity":10}'
for i in $(seq 1 40); do ( curl -X POST /api/orders ... ) & done; wait
```

**Result:**

```
orders:      18 PAID • 22 CANCELLED
inventory:   product_id=2 | quantity=10 | reserved=7 | version=29
```

| Expected        | Actual                   |
|-----------------|--------------------------|
| ≤10 orders PAID | <b>18 PAID</b>           |
| quantity → 0    | <b>quantity still 10</b> |

| Expected                       | Actual       |
|--------------------------------|--------------|
| reserved consistent with sales | reserved = 7 |

**8 units oversold, and stock never decreased at all.**

## Cause 1 — the Redis lock is released before the transaction commits

```
InventoryService.reserveStock() :  
  
@Transactional // ← transaction spans the method  
public boolean reserveStock(Long productId, Integer quantity, Long orderId) {  
    Boolean locked = redisTemplate.opsForValue().setIfAbsent(lockKey, ...);  
    try {  
        Inventory inventory = inventoryRepository.findByProductId(productId)...;  
        int available = inventory.getQuantity() - inventory.getReserved();  
        if (available < quantity) return false;  
        inventory.setReserved(inventory.getReserved() + quantity);  
        inventoryRepository.save(inventory); // ← NOT committed yet  
        return true;  
    } finally {  
        redisTemplate.delete(lockKey); // ← lock released HERE  
    }  
} // ← commit happens HERE
```

With `@Transactional`, `save()` only flushes on **method exit** — which is *after* `finally`. So:

```
Thread A: acquire lock → read reserved=0 → set 1 → save (uncommitted) → RELEASE LOCK  
Thread B:          acquire lock → read reserved=0 ← A's write not visible yet  
          → set 1 → save → release  
both commit → two reservations, one unit consumed
```

The lock guards the read-modify-write **but not the commit**, so it does not actually serialise anything. `@Version` caught some of the collisions — hence `version=29` and 22 cancellations — but that is optimistic locking papering over a broken mutual exclusion, not the lock working.

**Fix:** release the lock *after* commit, not in `finally` inside the transaction. Either move the locking to a caller outside the `@Transactional` boundary, or use

`TransactionSynchronizationManager.registerSynchronization(...)` to delete the key in `afterCommit`. Simplest robust alternative: drop the Redis lock entirely and do the reservation atomically in SQL:

```
UPDATE inventory  
SET reserved = reserved + :qty  
WHERE product_id = :pid  
AND quantity - reserved >= :qty
```

...then treat `rowsAffected == 0` as "insufficient stock". The database enforces it in one statement; no distributed lock needed.

## Cause 2 — reservations are never converted into a deduction

---

`inventory-service` consumes **only one topic**:

```
@KafkaListener(topics = "product.created")
```

There is **no** `payment.processed` **consumer**. So when payment succeeds, nothing deducts stock.

The method to do it exists and is correct:

```
@Transactional
public void confirmStock(Long productId, Integer quantity) {
    inventory.setReserved(Math.max(0, inventory.getReserved() - quantity));
    inventory.setQuantity(inventory.getQuantity() - quantity); // the actual deduction
    inventoryRepository.save(inventory);
}
```

...but it is **only ever called from a unit test** — `InventoryServiceTest:103`. Never from production code.

That is why `quantity` stayed at 10 through 18 paid orders: the saga reserves, and then nothing ever commits the sale.

**Fix:** add the missing consumer.

```
@KafkaListener(topics = "payment.processed", groupId = "inventory-service")
public void onPaymentProcessed(JsonNode event) {
    if (event.get("success").asBoolean()) {
        for (JsonNode item : event.get("items")) {
            inventoryService.confirmStock(item.get("productId").asLong(),
                                          item.get("quantity").asInt());
        }
    }
}
```

The saga's compensation path is wired ( `order.cancelled` → release), but the **success** path is not. Failure was tested; success was not.

---

## 4. Latency

---

40 concurrent orders through a single `kubectl port-forward` :

| Order accepted ( <code>202</code> -style PENDING) | ~0.13s |
|---------------------------------------------------|--------|
| Saga to <code>PAID</code>                         | ~8s    |

`port-forward` is a single-connection tunnel and is itself a bottleneck — these are indicative, not a throughput benchmark. A real load test should run inside the cluster or through the ALB.

---

# Findings

---

| # | Finding                                                         | Severity | Fix                                                                       |
|---|-----------------------------------------------------------------|----------|---------------------------------------------------------------------------|
| 1 | Overselling — 18 units sold from 10                             | ●        | atomic conditional <code>UPDATE</code> , or release the lock after commit |
| 2 | <code>confirmStock()</code> never called — stock never deducted | ●        | add the <code>payment.processed</code> consumer                           |
| 3 | Kong OOMKilled at 512Mi                                         | ●        | raise to 1Gi ( <b>applied</b> )                                           |
| 4 | Optimistic-lock failures surface as cancelled orders            | ●        | retry on <code>OptimisticLockException</code> instead of cancelling       |

---

## 5. Fixes applied and re-verified

---

### 5.1 Oversell — atomic reservation

---

`InventoryRepository` — the check and the increment are now one statement, so the database serialises concurrent callers. The Redis lock is gone entirely: a lock cannot be released too early if there is no lock.

```
@Modifying
@Query(value = "UPDATE inventory SET reserved = reserved + :qty, version = version + 1 " +
    "WHERE product_id = :productId AND quantity - reserved >= :qty",
    nativeQuery = true)
int reserveAtomic(@Param("productId") Long productId, @Param("qty") Integer qty);

reserveStock() now treats rowsAffected == 0 as insufficient stock.
```

## 5.2 Missing deduction — the `payment.processed` consumer

---

`payment.processed` carries only `orderId/userId/amount/success` — **no items** — so inventory cannot learn from the event what to deduct. It now records its own reservations when reserving:

```
redisTemplate.opsForHash().increment("inventory:reservation:" + orderId,
                                     productId.toString(), quantity);
```

...and consumes the event that was previously unhandled:

```
@KafkaListener(topics = "payment.processed", groupId = "inventory-service-group", concurrency = "6")
public void handlePaymentProcessed(String message) {
    if (success) inventoryService.confirmOrder(orderId); // deducts quantity
    else         inventoryService.releaseOrder(orderId); // gives it back
}
```

Both paths delete the Redis key afterwards, so a Kafka **redelivery finds nothing and is a no-op** — idempotent by construction.

`order.cancelled` now also releases by `orderId` rather than replaying the event's items, for the same reason.



## 5.3 Kong memory — in the manifest, not just the cluster

---

`k8s/kong/kong.yaml` : `limits.memory 512Mi → 1Gi` , `requests 256Mi → 512Mi` . The earlier fix was applied live only; it would have been lost on the next apply.

## 5.4 Re-test — same conditions, 10 units, 40 concurrent orders

|                  | Before                   | After |
|------------------|--------------------------|-------|
| Orders PAID      | 18 ●                     | 10 ✓  |
| Orders CANCELLED | 22                       | 30    |
| quantity         | 10 — never decremented ● | 0 ✓   |
| reserved         | 7 — inconsistent ●       | 0 ✓   |

**Exactly 10 units sold from 10 units of stock.** The other 30 were correctly rejected, and stock now actually depletes.

## 5.5 Tests

The old tests asserted the lock behaviour and no longer applied. Replaced with 11 tests, including:

| Test                                               | Guards against                                                                               |
|----------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>reserveStock_neverDoesReadModifyWrite</code> | the exact oversell regression — asserts <code>findByProductId / save</code> are never called |
| <code>confirmOrder_deductsEveryReservedItem</code> | the missing deduction                                                                        |
| <code>confirmOrder_isIdempotentOnRedelivery</code> | Kafka redelivery double-deducting                                                            |
| <code>releaseOrder_isIdempotentOnRedelivery</code> | same on the failure path                                                                     |

Tests run: 11, Failures: 0, Errors: 0

The JaCoCo coverage gate **failed** on the first build after the fix — the new methods had no tests. That gate was working as intended, and the build passes with it enabled now.

## 6. Security hardening — applied and verified

### 6.1 NetworkPolicy — default-deny, per-service database access

**New:** `k8s/security/networkpolicy.yaml` — 17 policies.

The databases here are **pods**, so the rules use `podSelector` and can be far tighter than a security group ever could: **each service reaches only its own database**.

| Service              | May reach                        |
|----------------------|----------------------------------|
| user-service         | postgres-user, redis             |
| product-service      | postgres-product, redis, kafka   |
| order-service        | postgres-order, kafka            |
| inventory-service    | postgres-inventory, redis, kafka |
| payment-service      | postgres-payment, kafka          |
| notification-service | kafka only — it owns no database |

`order-service` **cannot** reach `postgres-user`. That is the database-per-service boundary enforced by the network, not just by convention.

**Attacker test** — an unprivileged busybox pod:

| Target                | Before    | After     |
|-----------------------|-----------|-----------|
| postgres-user:5432    | reachable | ✅ blocked |
| postgres-payment:5432 | reachable | ✅ blocked |
| redis:6379            | reachable | ✅ blocked |
| kafka:9092            | reachable | ✅ blocked |
| 1.1.1.1:443 (egress)  | reachable | ✅ blocked |

**Application test** — a default-deny rollout that breaks the app is not a fix:

```
login      ✅ 200      (user-service → postgres-user + redis)
product    ✅ created  (product-service → postgres-product + kafka)
restock     ✅ 200      (inventory-service → postgres-inventory)
order      ✅ 200 → PAID  full Kafka saga across 4 services
inventory  5 → 3      stock deducted correctly
```

Verified enforced on k3s (Flannel + kube-router policy controller). On EKS the VPC CNI needs its network-policy agent enabled, or Calico/Cilium.

## 6.2 Sealed Secrets — encrypted at rest, safe to commit

---

A Kubernetes `Secret` is base64 — **encoding, not encryption**. `secrets.yaml` therefore could never be committed.

Installed the Bitnami sealed-secrets controller and encrypted with the cluster's public key:

```
kubeseal --controller-namespace kube-system \  
--controller-name sealed-secrets-controller \  
--format yaml < k8s/services/secrets.yaml > k8s/security/sealed-secrets.yaml
```

```
kind: SealedSecret  
  JWT_SECRET: AgCm9PAq+17x7tzdXAE8zWiwn9snV+J/A07hGyL5yuJL...  
  USER_DB_PASS: AgBR1Kp0tUEN1eJ/IboMirL6Rf+03f0qaVzIR3Ky0FnW...
```

Only the controller's **private key inside the cluster** can decrypt this, so the file is safe in Git.

**Verified:** deleted the plaintext `Secret`, applied the `SealedSecret`, the controller regenerated the real `Secret`, and `login` still returned **200**.

## 6.3 The data-tier question — resolved

---

`scripts/01-vpc.sh` creates data subnets and a `db-sg` that nothing uses: the datastores are StatefulSets running as pods in the **private** subnets, and `db-sg` is never attached to anything.

**Decision: accept the 2-tier deployment and enforce isolation in Kubernetes** — which §6.1 now does, and does more strictly than `db-sg` would have. `db-sg` allowed *any* pod carrying `eks-sg` to reach *any* database on 5432; the policies allow only `order-service` → `postgres-order`, and so on. Security groups filter at the ENI level and cannot express per-pod rules.

**Follow-up:** remove the unused data subnets and `db-sg` from `01-vpc.sh`, or fix the comments. Leaving them implies a boundary the deployment does not have.

---

## Why the existing test suite missed this

---

`InventoryServiceTest` calls `confirmStock()` directly and asserts it deducts correctly — so the **method** is tested, but nothing tests that it is **reachable**. And `reserveStock()` is tested single-threaded, where releasing the lock early is invisible.

Both bugs need the two tests that were flagged as missing earlier:

- **concurrency test** — N threads, 1 unit of stock, assert exactly one wins
- **saga end-to-end test** — place an order, let it reach `PAID`, assert `quantity` actually decreased

The 40-request run above is effectively the first of those, executed by hand.

---

## 7. Authentication — the largest hole found

---

Five of the six services had no `SecurityConfig` and no filter of any kind. Only `user-service` had either. Kong carried rate-limiting, correlation-id and prometheus — but no `jwt` plugin. The token was minted and then never checked by anything.

| Request                                               | Before     | After |
|-------------------------------------------------------|------------|-------|
| <code>POST /api/orders</code> — no token              | <b>200</b> | 403   |
| <code>POST /api/orders</code> — garbage token         | <b>200</b> | 403   |
| <code>GET /api/orders/user/1</code> — no token        | <b>200</b> | 403   |
| <code>POST /api/products</code> — no token            | <b>200</b> | 403   |
| <code>POST /api/inventory/5/restock</code> — no token | <b>200</b> | 403   |

The restock one is the expensive one: an anonymous caller could set any product to any quantity.

Each service now verifies the signature itself rather than trusting the gateway. That is deliberate — Kong is not the only route to a pod, because `kubect1 port-forward` goes straight past it.

## 7.1 Broken object-level authorisation (OWASP API #1)

---

`order-service` read `userId` out of the request body:

```
POST /api/orders    Authorization: Bearer <chaos1's token>
{"userId": 1, "shippingAddress": "attacker", ...}
```

```
-> 200 {"id":83,"userId":1,...}    order booked against a different user
```

The JWT carried only an email and a role, so the service had no way to know who was calling. It now carries a `userId` claim, and `CreateOrderRequest` has no `userId` field at all.

| Attack, using a valid token of the attacker's own | Before | After          |
|---------------------------------------------------|--------|----------------|
| read another user's order                         | 200    | <b>404</b>     |
| list another user's orders                        | 200    | <b>403</b>     |
| cancel another user's order                       | 200    | <b>404</b>     |
| body <code>userId</code> pointing at someone else | obeyed | <b>ignored</b> |

404 rather than 403 on the single-order read is deliberate: 403 confirms the id is real, which is all that is needed to walk the order table one id at a time.



## 7.2 Still open

---

**Logout is only enforced at user-service.** It deletes the Redis session, so user-service rejects the token at once — the other five check signature and expiry only, and keep accepting it for the remaining 24h. Closing that needs a shared revocation check, or short-lived tokens with refresh.

---

## 8. Saga replies under burst — 4 orders silently stranded

---

20 concurrent orders against 7 units:

```
PAID=7  CANCELLED=9  PENDING=4
```

The four never reached a terminal state. What made this hard to see is that every metric said the system was healthy:

```
21 orders in the database
21 order.created events in Kafka      nothing lost
21 inventory.updated replies in Kafka  every order got an answer
consumer lag 0                      every reply was consumed
0 log lines mentioning orders 13-16   nothing complained
```

Orders 13, 14, 15 and 16 were created at `01:17:04.654`, `.708`, `.831`, `.831` — 180ms apart, at the peak of the burst.

**Cause.** `order.created` was published from inside the `@Transactional` method. The Kafka send goes out immediately; the commit happens when the method returns. `inventory-service` rejected those four within milliseconds and replied *before the row was visible*, and the handler did:

```
Order order = orderRepository.findById(orderId).orElse(null);
if (order == null || order.getStatus() != PENDING) {
    return; // no log, nothing
}
```

There are exactly two silent-return paths, and the database ruled the second one out — those orders *were* PENDING. So `order == null` : the reply arrived before the order existed.

**Fix.** Domain events go through Spring and are relayed to Kafka on `AFTER_COMMIT`, so a reply cannot arrive before the order it refers to. The null branches log an error instead of returning silently.

**Re-run after the fix:** `PAID=7 CANCELLED=13 PENDING=0`, and zero `unknown orderId` errors.

## 8.1 The money version of the same bug

---

Here nobody was charged: those four failed at inventory, and payment-service skips explicitly on `success=false`. The payments table held 7 rows for 7 PAID orders.

But `markPaid` had the identical pattern. Had the dropped reply been a `payment.processed`, the customer would have been **charged with the order still PENDING** — and the stale-order reaper would then have cancelled it. There is no refund path anywhere in the system. It did not happen because the payment route has an extra hop and therefore more slack. That is timing, not a guarantee.

## 8.2 acks=all is currently a no-op on this cluster

Producers were on the defaults ( `acks=1` , no idempotence) and are now `acks=all` + `enable.idempotence` . That is the right setting, but worth being precise about: every topic here is `ReplicationFactor: 1` with `min.insync.replicas=1` , so `acks=all` is presently identical to `acks=1` . It only pays off with more than one broker.

## 9. ALB target sync — tested against a real ALB

`05-deploy.sh` used to snapshot Kong's pod IPs once with `aws elbv2 register-targets` . That is correct exactly until the first pod restart; after it, the registered IP is dead and the replacement is unknown to the ALB.

Replaced with a `TargetGroupBinding` , then verified end to end against a Floci ALB ( `ecommerce-alb` , `active` , target-type `ip` ):

```
BEFORE targets: 10.42.0.40 10.42.0.61 10.42.0.62
KILL    kong pod 10.42.0.62
t+=15s  pods=[10.42.0.40 10.42.0.61 10.42.0.67]
        targets=[10.42.0.40 10.42.0.61 10.42.0.67]
```

It deregistered the dead IP and registered the new one on its own, in about 15 seconds.

**The binding was wrong on the first attempt, and it failed quietly.** `serviceRef.port` was `8000` — the container port — where the Service port (80) is required. The controller reported `Successfully reconciled` while logging `BackendNotFound: unable to find port 8000 on service ecommerce/kong-proxy` , and the target group stayed empty. Only looking at the actual targets caught it.

**Two Floci limits, not code problems:** targets read `unhealthy` because k3s pods live on `10.42.x` , outside the emulated VPC; and the controller cannot manage security-group rules because `providerID k3s://...` is not an EC2 instance. Both behave differently on real EKS, where the VPC CNI gives pods real subnet addresses.

## 10. The deploy scripts, run from a clean slate

Running `01..05` against an empty account for the first time exposed a family of bugs sharing one cause: the scripts assumed a fresh account, and `set -e` turned every "already exists" into a silent early exit.

| Script                 | Bug                                                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>01-vpc.sh</code> | <code>--association-id</code> given a subnet id; the NACL was created but never attached, and the script died there  |
| <code>03-ecr.sh</code> | <code>create-repository</code> fatal on re-run — nothing was built or pushed                                         |
| <code>03-ecr.sh</code> | git SHA used as the tag on a dirty tree; with <code>IfNotPresent</code> a node keeps the old image under the new tag |

| Script                    | Bug                                                                                                                                                           |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>04-eks.sh</code>    | IAM roles, both node groups and the cluster had no existence guards                                                                                           |
| <code>04-eks.sh</code>    | <code>delete-cluster</code> is async — it landed <i>after</i> the replacement was created and tore it down ~20s later, visible only as an unexplained SIGTERM |
| <code>04-eks.sh</code>    | the registry mirror pointed at a container name on Docker's default bridge, which has no DNS, so every pull failed <code>no such host</code>                  |
| <code>05-deploy.sh</code> | applied <b>no</b> security manifests — a clean run produced a cluster with zero NetworkPolicies                                                               |
| <code>05-deploy.sh</code> | <code>sed -i</code> on a tracked file, substituting a placeholder that file does not contain                                                                  |

After the fixes, `05-deploy.sh` exits 0 with 17 network policies, 1 PDB, and an ALB target group that tracks Kong automatically.

## 11. CI

The smoke test ran entirely without a token, so it would have passed against every hole in §7. It now runs as three principals — a user, an admin, and a second ordinary user for the cross-user checks — and asserts the denials, which is the regression nothing else here would notice.

### 29/29 pass in CI.

The first CI run after the auth change failed 7 checks, all downstream of one line that reported success:

```
✓ ADMIN token obtained (role=USER)
```

The promotion ran `psql -U postgres`, but CI builds `app-secrets` from GitHub secrets and the database user is whatever `USER_DB_USER` holds. `psql` failed, the error was discarded by `2>&1 >/dev/null`, and the script carried on with an ordinary token. The db user is now read from the same secret the services use, the `psql` error is printed, and the check requires the token to actually carry `ROLE_ADMIN` rather than merely to exist.

## 12. What this is NOT — remaining gaps

Everything above was verified on Floci. **It would not deploy to real AWS as it stands.**

### Blockers

- **No EBS CSI driver and no StorageClass.** PVCs bind to k3s `local-path`, which does not exist on EKS. All seven StatefulSets would sit `Pending`.
- **No IRSA for the AWS Load Balancer Controller.** It was run here with static credentials. Real AWS needs an OIDC provider, an IAM policy and a service account annotation — none of which the scripts create.
- `06-teardown.sh` **has never been run.** Suspected NAT/EIP leak, which on real AWS costs money quietly.

### Known weaknesses

- Single NAT gateway — one AZ failure cuts the whole private tier off the internet.
- Six services run `replicas: 1` here to fit the cluster's RAM. EKS should run two across two AZs, with the budget shape already in `pdb.yaml`.
- The ACM certificate is issued for a `.local` domain, which real ACM will not validate.
- Subnet discovery tags ( `kubernetes.io/role/elb` ) not verified.
- `metrics-server` is bundled by k3s but not by EKS, and HPA depends on it.
- Logout is not enforced outside user-service (\$7.2).
- No transactional outbox — the residual event-loss window is covered by a reaper, not eliminated (\$8).
- Five services still have thin unit-test coverage, and there are no Testcontainers integration tests.
- No distributed tracing or correlation-id propagation through Kafka headers.